Projet Allstate — l'idée centrale et les trois blocs

Guide d'étude pour Cadmon HOUNSOU · à lire avec `pipeline.py` · sujet 04, estimation du montant d'un sinistre

Partie 1 — Le raisonnement pas à pas, pour débutants

Cette partie explique pourquoi on fait chaque chose, avec des exemples de la vie courante et sans jargon. Lisez-la en premier ; la Partie 2 reprend les mêmes idées avec le vocabulaire exact et le lien avec le code.

Avant de commencer : ce que vous faites vraiment

Imaginez un assureur qui a déjà réglé 150 654 sinistres. Pour chacun, il a noté 130 renseignements sur le dossier, et le montant qu'il a fini par payer. Il reçoit maintenant 37 664 nouveaux dossiers : mêmes 130 renseignements, mais **sans le montant**. Il doit deviner ce qu'il paiera.

Votre travail : **apprendre des anciens dossiers pour deviner les nouveaux**. C'est tout l'apprentissage automatique de ce projet. L'enseignant garde les vrais montants des 37 664 nouveaux dossiers, comme le corrigé d'un examen, et vous note en comparant vos devinettes à ces vrais montants.

Les cinq mots à connaître :

- **Ligne** : un sinistre. **Colonne (variable)** : un renseignement sur le sinistre.
- **Cible** : ce qu'on veut deviner, ici `loss`, le montant.
- **Modèle** : une machine à deviner, qui a appris à partir d'exemples.
- **Entraîner** : laisser le modèle étudier des exemples dont on connaît le montant.
- **Prédire** : demander au modèle de deviner le montant d'un dossier.

Le fil conducteur tient en une phrase : **avant de chercher à deviner bien, il faut savoir comment on mesure « bien »**. D'où les trois niveaux qui suivent : mesurer (Bloc 1), construire (Bloc 2), vérifier (Bloc 3).

Niveau 1 — Mesurer avant de construire (Bloc 1)

Pourquoi commencer par la mesure ? Si vous construisez un modèle sans savoir comment on vous note, vous ne saurez jamais si vous progressez. C'est comme préparer un examen sans connaître le barème.

Étape 1 — Choisir la note : la MAE. Pour chaque sinistre, on calcule l'écart entre votre devinette et le vrai montant (peu importe le sens), puis on fait la moyenne de ces écarts. Exemple : trois sinistres de 1 000, 2 000 et 10 000 €. Vous devinez 2 000 € pour tous. Les écarts sont 1 000, 0 et 8 000, donc la MAE vaut 3 000 €. Plus elle est petite, mieux c'est. Elle est en euros : c'est directement un coût, ce qui répond à la question d'oral « pourquoi cette métrique ? ».

Étape 2 — Que deviner quand on n'a aucun modèle ? Si vous deviez annoncer **un seul montant** pour tous les dossiers, lequel choisir ? C'est la constante de référence. Prenons cinq sinistres : 500, 800, 1 000, 1 200 et 50 000 €.

Vous annoncez toujours	Écarts avec les 5 vrais montants	MAE
La moyenne (10 700 €)	10 200 ; 9 900 ; 9 700 ; 9 500 ; 39 300	15 720 €
La médiane (1 000 €)	500 ; 200 ; 0 ; 200 ; 49 000	9 980 €

La **médiane** est la valeur du milieu quand on range les montants du plus petit au plus grand. Un seul énorme sinistre tire la moyenne très haut : elle ne ressemble alors à aucun sinistre ordinaire. La médiane reste au milieu. Avec la MAE, la médiane gagne. Le sujet part d'une MAE de 1 807,09 en annonçant la moyenne : votre Bloc 1 mesure ce que donne la médiane sur les vraies données.

Cette devinette sans modèle est votre **plancher**. Un modèle qui ne la bat pas ne sert à rien.

Étape 3 — Le coût des erreurs. Se tromper n'a pas forcément le même prix dans les deux sens. Sous-estimer, c'est mettre trop peu d'argent de côté et risquer de ne pas pouvoir payer. Surestimer, c'est immobiliser de l'argent pour rien. Question à trancher en équipe : l'un des deux est-il plus grave, et de combien de fois ? Il n'y a pas de bonne réponse imposée : vous choisissez et vous expliquez. Le code montre comment la devinette « penche » quand sous-estimer coûte plus cher.

Étape 4 — Tester honnêtement : la validation. Règle d'or : ne jamais noter un modèle sur les exemples qui lui ont servi à apprendre, car il peut les avoir appris par cœur. C'est comme s'entraîner avec des annales : pour savoir si l'on a compris, on cache quelques exercices.

On coupe donc les 150 654 sinistres en 5 paquets. On entraîne sur 4 paquets et on teste sur le 5e, que le modèle n'a jamais vu. Puis on recommence en cachant un autre paquet, cinq fois. À la fin, chaque sinistre a été deviné une fois sans que le modèle l'ait vu.

Pourquoi des paquets « stratifiés » ? Quelques sinistres sont énormes (le plus gros dépasse 121 000 €). Si le hasard met presque tous les gros dans un seul paquet, la note de ce paquet est mauvaise et celle des autres est bonne : la note dépend de la chance. On

répartit donc volontairement : chaque paquet reçoit autant de petits, de moyens et de gros sinistres.

Ce que vous écrivez pour le Bloc 1 : un paragraphe par étape, avec vos propres chiffres. Pour les obtenir : `python pipeline.py bloc1`.

Niveau 2 — Construire (Bloc 2)

Comment le modèle devine. Pensez au jeu des vingt questions : « `cat1` vaut-il A ? », « `cont2` dépasse-t-il 0,4 ? », et ainsi de suite, jusqu'à arriver à une devinette. Cette suite de questions s'appelle un **arbre de décision**. Un seul arbre est trop simple. On en construit donc des centaines, l'un après l'autre, chacun corrigeant les erreurs des précédents, comme une équipe de correcteurs qui se passent la copie. C'est le principe du « boosting », utilisé ici via `HistGradientBoostingRegressor` de scikit-learn. Vous n'avez pas besoin d'en connaître les maths : vous devez comprendre **ce qu'on lui demande** et **ce qu'on lui donne**.

Ce qu'on lui demande : la « perte ». On dit au modèle quelle erreur il doit chercher à réduire. Si on lui demande l'erreur absolue (la MAE), il vise la médiane pour des dossiers qui se ressemblent. Si on lui demande l'erreur au carré, il vise la moyenne et se laisse attirer par les gros sinistres. Votre note est la MAE : on lui demande donc la MAE. Le principe à retenir : **ce qu'on demande au modèle doit correspondre à la façon dont on est noté**.

Le logarithme, sans peur. Les montants vont de moins de 1 € à plus de 100 000 €. Le logarithme revient à compter en ordres de grandeur : 100, 1 000, 10 000 et 100 000 deviennent des marches égales. Le modèle apprend plus facilement sur ces nombres rapprochés, puis on repasse en euros pour la réponse. Détail rassurant : le logarithme ne change pas la médiane, donc on ne trahit pas la note.

Transformer les lettres en nombres : l'encodage. Les 116 variables `cat` contiennent des lettres (`A`, `B`, `AF`…), et un modèle ne calcule qu'avec des nombres. Plusieurs méthodes existent :

Méthode	L'idée	L'image	Point d'attention
Ordinal	chaque valeur reçoit un numéro	numéroter des élèves par ordre alphabétique	l'ordre est arbitraire
Fréquence	chaque valeur devient sa fréquence	remplacer un prénom par « combien d'élèves le portent »	on perd l'identité exacte

Méthode	L'idée	L'image	Point d'attention
Cible	chaque valeur devient le montant moyen de ceux qui l'ont	remplacer un quartier par le prix moyen de ses maisons	triche possible si mal placé (voir Niveau 3)
Natif	le modèle comprend les catégories	lui laisser lire les mots	limité à 255 valeurs par variable
One-hot	une colonne oui/non par valeur	une case à cocher par valeur	plus de 1 000 colonnes ici

On ne sait pas d'avance laquelle est la meilleure sur ces données. C'est la raison d'être des essais.

Les essais. Règle d'or : **changer une seule chose à la fois**, sinon on ne sait pas laquelle a fait bouger le résultat. Les essais E01 à E11 changent chacun un point : la constante, la perte, la cible brute ou logarithmique, l'encodage, les réglages. Pour lire le journal (`sorties/journal_essais.csv`) :

- `mae` : plus petit = mieux. Comparez toujours à E02, la médiane, votre plancher.
- `gain_vs_mediane_pct` : de combien de pour cent vous battez ce plancher. C'est la réponse chiffrée à la question du sujet.
- `mae_ecart_type_plis` : le « bruit » naturel entre paquets. Si deux essais diffèrent de moins que ce bruit, considérez-les à égalité.

Gardez **tous** les essais, y compris les ratés : un échec qu'on sait expliquer (comme E04, qui vise la moyenne alors qu'on est noté sur la médiane) est une information utile, et c'est un critère de notation.

L'analyse d'erreurs : où le modèle se trompe-t-il ? Un modèle a vu peu de très gros sinistres, il les devine donc mal. Regardez : quelle part de l'erreur totale vient du 1 % des plus gros sinistres ? Le modèle sous-estime-t-il plutôt qu'il ne surestime ? Votre phrase à produire : « le modèle rate surtout tels dossiers, et cela coûte tant ».

Pour le Bloc 2 : commencez par `ECHANTILLON=30000 python pipeline.py bloc2` pour vérifier que tout marche, puis lancez la version complète, qui est longue.

Niveau 3 — Vérifier que c'est fiable (Bloc 3)

Le test de robustesse : et si le hasard tombait autrement ? Une partie de l'apprentissage dépend du hasard (comment on forme les paquets, comment le modèle démarre). On refait donc le test trois fois avec un hasard différent, ce qu'on règle avec la « graine ». Si la note reste proche, le résultat est solide. Si elle bouge beaucoup, votre bon score n'était qu'un coup de chance, comme un dé lancé une seule fois.

Les trois mutations : et si on changeait une décision ? Une mutation est une expérience où l'on modifie **une seule ligne** de réglage pour voir l'effet. Il y en a trois :

- **M1 — et si on avait visé la moyenne ?** On change la décision sur la note. Le résultat montre qu'un choix qui paraît théorique modifie le modèle.
- **M2 — et si on codait autrement les lettres ?** On change l'encodage.
- **M3 — la triche volontaire.** On apprend l'encodage « cible » sur **toutes** les lignes, y compris celles qui servent ensuite à noter. C'est l'étudiant qui a lu le corrigé avant l'examen : sa note est excellente et ne prouve rien. Si M3 fait mieux que votre résultat de référence, c'est la preuve que cette « fuite » gonfle les notes. C'est pour cela qu'on découpe **avant** d'apprendre quoi que ce soit.

À l'oral, on vous demandera de montrer « la mutation qui casse le découpage » : c'est M3.

L'estimation finale. Le modèle retenu apprend sur tous les anciens sinistres, devine les 37 664 nouveaux et écrit `soumission.csv` avec deux colonnes, `id` et `loss`. Le programme chronomètre l'opération : elle doit rester sous 10 minutes sur un ordinateur ordinaire.

Après le Bloc 3 : il reste à écrire la note de 3 pages, la model card et la fiche de données. Ils s'écrivent à partir de vos résultats, pas avant.

Votre premier lancement, pas à pas

1. Téléchargez le dossier `Sujet_04_Allstate` (lien dans l'énoncé) et vérifiez qu'il pèse environ 70,3 Mo.
2. Placez `pipeline.py` dans ce dossier.
3. Ouvrez le dossier dans VS Code, puis ouvrez le terminal intégré.
4. Créez l'environnement : `python -m venv .venv`. Activez-le (`source .venv/bin/activate` sous Mac ou Linux, `.venv\Scripts\activate` sous Windows), puis installez les outils : `pip install pandas scikit-learn`.
5. Faites un test rapide sur 30 000 lignes. Sous Mac ou Linux : `ECHANTILLON=30000 python pipeline.py bloc1`. Sous Windows PowerShell : `$env:ECHANTILLON=30000; python pipeline.py bloc1`.
6. Si cela fonctionne, relancez sans `ECHANTILLON` : d'abord `bloc1`, puis `bloc2` (long, laissez tourner), puis `bloc3`.
7. Après chaque bloc, ouvrez le dossier `sorties/` et lisez les fichiers produits.

Si un message d'erreur apparaît, copiez-le en entier et envoyez-le moi : une erreur de débutant se règle en général en quelques minutes.

Comment lire vos résultats et quoi écrire

Bloc	Fichier à ouvrir	La question à vous poser	Ce que vous écrivez
1	affichage de `bloc1` et `bloc1_mae_selon_constante.csv`	Quelle constante est la meilleure, et de combien ?	La note choisie et pourquoi, la constante mesurée, le coût, le découpage
2	`journal_essais.csv`	Quel essai bat le plancher, de combien de pour cent, et lesquels ont échoué ?	Le tableau des essais commenté, l'encodage retenu et pourquoi
2	`erreurs_top10.csv` et `erreurs_par_decile.csv`	Quels sinistres le modèle rate-t-il, et que cela coûte-t-il ?	Une analyse d'erreurs chiffrée
3	`robustesse.csv` et `mutations.csv`	Le score tient-il quand le hasard change ? Que montre M3 ?	Les trois mutations et la conclusion sur la fiabilité

On ne vous demandera pas d'expliquer les mathématiques des arbres. On vous demandera de **défendre vos décisions** : pourquoi cette note, pourquoi ce découpage, pourquoi cet encodage, qui le modèle rate-t-il.

Chacun doit pouvoir défendre seul la décision de son rôle, donc relisez le niveau qui vous correspond : le **mesureur** relit le Niveau 1, le **constructeur** le Niveau 2 et les mutations du Niveau 3, l'**interprète** l'analyse d'erreurs du Niveau 2 et la robustesse du Niveau 3.

Quand vous aurez lancé les trois blocs, envoyez-moi vos fichiers de `sorties/` : nous écrirons ensemble la note, la model card et la fiche de données à partir de vos vrais chiffres.

Partie 2 — Référence technique

À lire après la Partie 1 : mêmes idées, avec le vocabulaire exact, le détail de chaque essai et le lien avec `pipeline.py`.

1. L'idée centrale, en clair

Un assureur reçoit un dossier de sinistre décrit par 130 variables anonymes (116 catégorielles `cat1…cat116`, 14 continues `cont1…cont14`). Il doit estimer **combien** il paiera (`loss`). On ne prédit pas une classe, on prédit un montant, et **l'erreur se paie en euros**.

La question officielle du sujet : *peut-on estimer ce montant à partir du dossier, et de combien une méthode soignée bat-elle l'estimation que l'assureur applique aujourd'hui sans modèle ?*

Ce sujet n'est donc pas un concours « qui a le plus gros modèle ». C'est un exercice de **jugement**, avec trois idées qui s'enchaînent :

- **La métrique décide de tout.** On choisit comment on sera jugé (ici l'erreur absolue moyenne, la MAE). Ce choix détermine ce qu'est une « bonne réponse » : la bonne constante sans modèle, puis la bonne cible pour le modèle.
- **La queue de distribution décide de qui est mal servi.** Quelques très gros sinistres (jusqu'à 121 012) pèsent énormément dans l'erreur totale. Un modèle peut sembler bon en moyenne et rater systématiquement les gros dossiers.
- **L'honnêteté de la mesure.** Un score n'a de valeur que s'il a été obtenu sur des données que le modèle n'a jamais vues, avec un découpage justifié et sans fuite d'information. Le journal doit garder les échecs.

Tout le reste (encodage, arbres, réglages) est de l'outillage au service de ces trois idées.

Pourquoi « moyenne contre médiane » revient sans cesse

La distribution des montants est très étirée vers le haut : peu de gros sinistres tirent la moyenne vers le haut, alors que la médiane reste au milieu des sinistres « ordinaires ». Avec la MAE, la constante unique qui minimise l'erreur est la **médiane**, pas la moyenne. L'énoncé annonce une MAE de 1 807,09 en prédisant la moyenne et parle d'un écart de 8,9 % entre moyenne et médiane : c'est ce que le Bloc 1 te fait mesurer toi-même, au lieu de le croire sur parole.

2. Le vocabulaire à maîtriser

- **MAE (erreur absolue moyenne)** : la moyenne de |prédit − réel|. Elle s'exprime en euros et traite une erreur de 1 000 € comme dix erreurs de 100 €.
- **Constante de référence** : la valeur unique qu'on annoncerait pour tous les sinistres si l'on n'avait aucun modèle. C'est le « plancher » à battre.
- **Médiane conditionnelle** : la médiane du montant *pour des dossiers qui se ressemblent*. Un modèle entraîné avec une perte absolue (`absolute_error`) cherche exactement cela.

- **Validation croisée** : on coupe `train.csv` en 5 plis ; on entraîne sur 4 et on prédit le 5e, à tour de rôle. On obtient une prédiction « honnête » pour chaque ligne.
- **Stratification** : on s'arrange pour que chaque pli contienne la même proportion de petits, moyens et gros sinistres.
- **Fuite (leakage)** : une information qui ne serait pas connue au moment où le dossier est instruit, ou qui vient de la cible des lignes de validation, et qui se glisse dans l'entraînement. Elle gonfle le score sans rien valoir en réalité.
- **Encodage** : transformer des lettres (`A` , `B` , `AF` …) en nombres que le modèle peut utiliser.
- **Mutation** : un changement d'une seule ligne de configuration (métrique, encodage, découpage) qui permet de comparer deux versions sans réécrire le pipeline.

3. Bloc 1 — la décision et la mesure

Question portée : sur quoi juge-t-on ? C'est le travail du mesureur.

3.1 La métrique

On adopte la **MAE**. Ce que cela implique : toutes les erreurs comptent proportionnellement à leur taille en euros, donc les gros sinistres pèsent lourd ; et le meilleur pari est la médiane. Une métrique quadratique (RMSE) pénaliserait encore plus les gros sinistres et viserait la moyenne : ce serait un autre problème. D'où la phrase à pouvoir dire à l'oral : « *je n'utilise pas l'exactitude parce qu'il n'y a pas de classe ; la MAE est en euros, donc directement lisible comme un coût.* »

3.2 La constante de référence

Sans modèle, quelle valeur unique annoncer ? Le code calcule la MAE pour toute une grille de constantes et montre que le minimum tombe sur la médiane. Il mesure ensuite les deux constantes avec le même protocole de validation que les modèles. La médiane est la référence à battre ; la moyenne ne sert que de point de comparaison avec l'énoncé.

3.3 Le coût de l'erreur

Avec la MAE, sous-estimer de 100 € et surestimer de 100 € coûtent la même chose. En réalité ce n'est pas symétrique : sous-estimer un sinistre, c'est constituer une provision insuffisante ; surestimer, c'est immobiliser trop d'argent. Le code simule un coût où la sous-estimation coûte *k* fois plus que la surestimation : la constante optimale devient alors le quantile k/(1+k) de la cible (k = 1 redonne la médiane). Si votre équipe décide que l'asymétrie existe, le modèle s'adapte avec `perte="quantile"` et `quantile=k/(1+k)` . À vous de décider et de justifier : le sujet ne donne pas la valeur de *k*.

3.4 Le protocole de validation

Validation croisée à **5 plis stratifiés sur la taille du sinistre** (déciles du montant, plus une tranche à part pour le 1 % le plus lourd).

Pourquoi ce découpage-là : la cible est un montant très asymétrique. Avec un découpage purement aléatoire, un pli peut recevoir presque tous les gros sinistres et le score varie surtout par chance. Le code affiche, pour les deux découpages, l'écart-type de la moyenne de `loss` d'un pli à l'autre : plus il est petit, plus les plis se ressemblent.

Règles associées : la colonne `id` n'est **jamais** une variable (c'est un numéro interne) ; tout ce qui est appris (encodage, modèle) l'est sur le pli d'entraînement seul.

3.5 À rédiger pour le Bloc 1

Un paragraphe par point : métrique et implication, constante mesurée (avec ton chiffre), coût, protocole et justification. Lance `python pipeline.py bloc1` et recopie tes propres chiffres.

4. Bloc 2 — le pipeline

Question portée : avec quoi, et en combien de temps ? C'est le travail du constructeur, avec l'analyse d'erreurs de l'interprète.

4.1 Le choix du modèle

Le pipeline utilise `HistGradientBoostingRegressor` de scikit-learn : des arbres de décision construits les uns après les autres, chacun corrigeant les erreurs des précédents. C'est rapide sur CPU, gère les valeurs manquantes et accepte une perte absolue. Il respecte la contrainte « scikit-learn seul, moins de 10 minutes ».

4.2 La cible : brute ou log ?

Les montants sont très étirés. On peut apprendre `log(1+loss)` puis revenir en euros. Point à comprendre : une transformation croissante comme le log ne change pas la médiane. Apprendre le log avec une perte absolue vise donc la même quantité que sur le montant brut, mais l'apprentissage est souvent plus facile. En revanche, avec une perte quadratique sur le log, on ne vise plus la médiane ni la moyenne : c'est pourquoi l'essai E07 sert de contre-exemple.

4.3 Les 116 variables catégorielles

Le one-hot (une colonne par modalité) donnerait plus de mille colonnes creuses : lourd et sans bénéfice net pour des arbres. Le code compare quatre encodages :

- **ordinal** : chaque modalité devient un numéro. Rapide, ordre arbitraire.
- **fréquence** : chaque modalité devient sa fréquence dans l'entraînement. Donne un sens « rare / courante » sans toucher à la cible.
- **cible** : chaque modalité devient la moyenne de la cible qui lui correspond. Puissant mais dangereux : appris au mauvais endroit, il provoque une fuite. Dans le pipeline, scikit-learn le protège (calcul croisé interne pendant l'entraînement, encodage du pli d'entraînement seul pour la validation).
- **natif** : le modèle traite la variable comme une vraie catégorie. Limite de scikit-learn : 255 modalités maximum par variable, donc les variables plus larges (comme celle à 326 modalités) restent en ordinal.

C'est exactement le sens de la remarque de l'énoncé : « l'encodage le plus évident n'est pas toujours tenable ».

4.4 Le tableau des essais

L'énoncé exige au moins huit essais, échecs compris, chacun avec son score. Le code en lance onze (E01 à E11) et écrit `sorties/journal_essais.csv` après chaque essai :

- E01 et E02 : les deux constantes (moyenne, médiane).
- E03 : modèle linéaire (Ridge sur one-hot), pour mesurer ce qu'apporte la non-linéarité.
- E04 à E07 : le croisement perte (quadratique ou absolue) × cible (brute ou log). E04 est un échec attendu par construction.
- E08 à E10 : les encodages fréquence, cible, natif.
- E11 : le meilleur schéma réglé plus finement.

Le journal contient aussi, pour chaque essai, l'écart-type entre plis, la MAE sur le 1 % de sinistres les plus lourds et le gain en pourcentage par rapport à la médiane : c'est la réponse chiffrée à la question du sujet.

Important : ne retouche pas un score pour qu'il soit joli. Un essai qui échoue reste dans le tableau avec son score. C'est un critère de notation (la vérité du journal).

4.5 L'analyse d'erreurs

Le code mesure : la part de l'erreur totale due au 1 %, 5 % et 10 % des sinistres les plus lourds ; la proportion de sous-estimations et le biais moyen ; les dix sinistres les plus lourds (réel, prédit, erreur) ; l'erreur par décile de montant. À interpréter dans ta note : qui le modèle rate-t-il le plus, et combien ça coûte ? Cherche surtout si les gros sinistres sont systématiquement sous-estimés (tendance habituelle en régression : les prédictions se tassent vers le milieu).

5. Bloc 3 — le dépôt et la robustesse

Questions portées : que vaut le résultat ? Et le code est-il réparable ?

5.1 Le test de robustesse

Le score de l'essai retenu est recalculé avec trois graines différentes (la graine change le découpage et le modèle). Si les trois MAE sont proches, le résultat ne dépend pas d'un coup de chance. Le code écrit `sorties/robustesse.csv`.

5.2 Les trois mutations nommées

Chacune est une modification d'une ligne de configuration, définie dans le dictionnaire `MUTATIONS` :

- **M1_metrique_moyenne** : on vise la moyenne (perte quadratique, montant brut) au lieu de la médiane. Montre que la décision métrique change le modèle.
- **M2_encodage_frequence** : on change l'encodage des catégorielles.
- **M3_fuite_encodage** : on **casse volontairement le découpage** en apprenant l'encodage « cible » sur toutes les lignes avant la validation. C'est la réponse à la question d'oral « montre-moi la mutation qui casse le découpage ». Si M3 fait *mieux* que la référence, c'est un score trop beau pour être vrai : la preuve qu'il est faux.

5.3 L'estimation finale

`python pipeline.py final` entraîne le meilleur modèle exportable sur tout `train.csv`, estime les 37 664 sinistres de `a_estimer.csv`, contrôle le format (`id,loss`, aucune valeur manquante) et écrit `soumission.csv`. Il chronomètre l'opération et signale un dépassement des 10 minutes. Par défaut, le choix de l'essai est automatique (le plus petit MAE du journal, parmi les essais exportables) ; pour imposer un essai, modifie la variable `CHOIX_FINAL` en haut du fichier.

5.4 Les livrables écrits

Le dépôt doit aussi contenir la note de 3 pages, la model card et la fiche de données du S13. Ils se rédigent à partir de tes résultats réels : voir la section 8.

6. Se repérer dans pipeline.py

Le fichier est découpé en sections numérotées, repérables par les titres entourés de lignes `=====` :

- **0. Réglages et CONFIG** : le seul endroit à modifier pour changer de version (graine, plis, perte, cible, encodage, paramètres du modèle).
- **1. Trouver et charger les données** : recherche automatique du dossier, contrôles d'hygiène (pas de manquants, colonnes cohérentes, `loss` absente de `a_estimer.csv`).
- **2. Encodage et modèle** : les quatre encodages, la construction du modèle, le modèle linéaire de comparaison.
- **3. Validation et journal** : le protocole (plis stratifiés), la fonction `valider`, l'écriture du journal.
- **4. Bloc 1**, **5. Bloc 2**, **6. Bloc 3** : une fonction par bloc.
- **7. Point d'entrée** : les commandes `bloc1`, `bloc2`, `bloc3`, `final`.

Comment l'exécuter

- Installer : `pip install pandas scikit-learn` (version 1.3 ou plus récente de scikit-learn, nécessaire pour l'encodage « cible »).
- Placer `pipeline.py` à côté du dossier des données, ou définir `DONNEES_PROJET`.
- Premier test rapide : `ECHANTILLON=30000 python pipeline.py bloc2` (30 000 lignes au lieu de 150 654).
- Puis les vraies exécutions, dans l'ordre : `bloc1`, `bloc2`, `bloc3`.
- Chronométrer l'estimation finale seule avec `final` : c'est la mesure qui compte pour la limite de 10 minutes.

7. Ce que je n'ai pas pu vérifier

- J'ai testé le code de bout en bout sur **des données synthétiques** de même structure (mêmes noms de colonnes, 116 catégorielles, 14 continues). Je n'avais pas ton vrai `train.csv`. Les scores de mes tests n'ont aucune valeur pour ton projet et les classements d'essais peuvent être différents sur les vraies données.
- Les durées sont à mesurer sur ta machine. Les 11 essais en 5 plis sur 150 654 lignes peuvent prendre longtemps (je m'attends à plusieurs dizaines de minutes sur un portable, estimation non mesurée) : utilise `ECHANTILLON` pour déboguer, puis lance la version complète une fois. L'estimation finale seule est bien plus courte, mais c'est à confirmer par une mesure.
- Les choix de l'équipe (valeur de *k* pour l'asymétrie, essai retenu, rôles) restent à décider et à justifier par vous.

8. Pièges à éviter

- Utiliser `id` comme variable : c'est un numéro interne, pas une information sur le sinistre.
- Apprendre quoi que ce soit (encodage, normalisation, choix de seuil) avant le découpage en plis.
- Juger un modèle sur le score d'entraînement : seule la prédiction hors pli compte.
- Comparer des essais avec des découpages ou des graines différents sans le dire.
- Supprimer du journal un essai qui a échoué.
- Annoncer des chiffres de ce guide ou de mes tests dans la note : seuls tes chiffres, issus de ton exécution, comptent.

9. Les questions d'oral, avec la réponse en une phrase

- « *Pourquoi cette métrique, et pas l'exactitude ?* » (mesureur) : il n'y a pas de classe à prédire, la MAE est en euros donc lisible comme un coût, et elle fixe la médiane comme bonne constante.
- « *Montre-moi la mutation qui casse le découpage.* » (constructeur) : M3, l'encodage « cible » appris avant la validation ; le score s'améliore artificiellement.
- « *Qui rates-tu le plus, et combien ça coûte ?* » (interprète) : les plus gros sinistres ; la part de l'erreur totale due au 1 % le plus lourd figure dans l'analyse d'erreurs, avec ton chiffre.
- « *Moyenne ou médiane ?* » : médiane, parce que la MAE est minimale à la médiane ; la différence mesurée est celle de ton Bloc 1.
- « *Pourquoi pas le one-hot ?* » : plus de mille colonnes creuses pour un gain nul avec des arbres ; les encodages alternatifs sont comparés dans E08, E09 et E10.

10. Prochaines étapes

- Lancer `bloc1`, puis `bloc2`, puis `bloc3` sur les vraies données.
- Garder `sorties/journal_essais.csv`, `sorties/erreurs_top10.csv`, `sorties/erreurs_par_decile.csv`, `sorties/robustesse.csv`, `sorties/mutations.csv`, et le temps affiché par `final`.
- Revenir avec ces résultats : on rédigera ensemble la note de 3 pages, la model card et la fiche de données à partir de tes chiffres réels.